

assignment_2

December 8, 2024

#

Assignment 2: Data Collection & Understanding

###

Chioma Onyekpere

###

Predictive Analytics Diploma, PACE, University of Winnipeg

###

Foundations of Data Science

###

Instructor: Muhammad Shahin

###

December 7, 2024

##

Employee Performance and Retention Analysis

0.1 Objectives

The primary goal of this report is to comprehensively analyze and prepare datasets for use in predictive modeling to improve employee retention and performance. This report focuses on the **Data Understanding Phase** of the CRISP-DM framework, aiming to:

1. **Assess Data Relevance:**
 - Confirm that the datasets align with the goals of predicting employee attrition and improving performance.
 - Ensure the datasets cover key attributes such as job satisfaction, performance ratings, and demographics.
2. **Evaluate Data Quality:**
 - Identify issues such as missing values, duplicate records, and inconsistencies.
 - Assess the completeness, accuracy, and usability of the data.
3. **Describe Data Characteristics:**
 - Provide an overview of the datasets, including size, structure, and key statistics.
 - Summarize relationships between variables relevant to the data mining goals.
4. **Prepare Data for Modeling:**
 - Resolve data quality issues through cleaning, standardization, and imputation.
 - Explore relationships and distributions to identify trends and anomalies.
5. **Document Findings:**
 - Ensure transparency by documenting data sources, quality issues, and resolutions.
 - Provide insights into the impact of data quality on the predictive modeling phase.

By addressing these objectives, this report ensures that the datasets are reliable, clean, and ready for the next phases of analysis and modeling. ## 1. Data Collection

0.1.1 Overview

The datasets used for this study were acquired from Kaggle's **HR Analytics Employee Attrition & Performance** repository. The files include information on employee demographics, job performance, satisfaction, education, and more. These datasets provide a comprehensive foundation for exploring the factors affecting employee retention and performance.

0.1.2 Datasets

The following datasets were downloaded for analysis: 1. **Employee.csv**: Employee demographic and job-related details. 2. **PerformanceRating.csv**: Historical performance evaluations. 3. **SatisfiedLevel.csv**: Satisfaction levels mapped to descriptive categories. 4. **EducationLevel.csv**: Educational qualifications. 5. **RatingLevel.csv**: Performance rating categories. 6. **Dim-Date.txt**: Calendar-related information for time-series analysis.

Dataset Name	Location	Acquisition Method	Data Issues Noted	Resolutions Achieved
Employee.csv	Local folder	CSV	None identified	N/A
DimDate.txt	Local folder	Text File	None identified	N/A
PerformanceRating.csv	Local folder	CSV	None identified	N/A
RatingLevel.csv	Local folder	CSV	None identified	N/A
EducationLevel.csv	Local folder	CSV	None identified	N/A
SatisfiedLevel.csv	Local folder	CSV	None identified	N/A

```
[17]: # Import necessary libraries
import pandas as pd
import numpy as np

# Load datasets
df_employee = pd.read_csv('Employee.csv')
df_performance = pd.read_csv('PerformanceRating.csv')
df_satisfaction = pd.read_csv('SatisfiedLevel.csv')
df_education = pd.read_csv('EducationLevel.csv')
df_rating_level = pd.read_csv('RatingLevel.csv')
df_date = pd.read_csv('DimDate.txt', delimiter='\t')

# Confirm dataset structures
print("Employee Dataset:\n", df_employee.head())
```

Employee Dataset:

	EmployeeID	FirstName	LastName	Gender	Age	BusinessTravel	\
0	3012-1A41	Leonelle	Simco	Female	30	Some Travel	
1	CBCB-9C9D	Leonerd	Aland	Male	38	Some Travel	
2	95D7-1CE9	Ahmed	Sykes	Male	43	Some Travel	
3	47A0-559B	Ermentrude	Berrie	Non-Binary	39	Some Travel	
4	42CC-040A	Stace	Savege	Female	29	Some Travel	

	Department	DistanceFromHome	(KM)	State	Ethnicity	...	\
0	Sales		27	IL	White	...	
1	Sales		23	CA	White	...	
2	Human Resources		29	CA	Asian or Asian American	...	
3	Technology		12	IL	White	...	
4	Human Resources		29	CA	White	...	

	MaritalStatus	Salary	StockOptionLevel	OverTime	HireDate	Attrition	\
0	Divorced	102059		1	No	2012-01-03	No

1	Single	157718	0	Yes	2012-01-04	No
2	Married	309964	1	No	2012-01-04	No
3	Married	293132	0	No	2012-01-05	No
4	Single	49606	0	No	2012-01-05	Yes

	YearsAtCompany	YearsInMostRecentRole	YearsSinceLastPromotion	\
0	10	4	9	
1	10	6	10	
2	10	6	10	
3	10	10	10	
4	6	1	1	

	YearsWithCurrManager
0	7
1	0
2	8
3	0
4	6

[5 rows x 23 columns]

```
[18]: # Confirm dataset structures
print("Performance Rating Dataset:\n", df_performance.head())
```

Performance Rating Dataset:

	PerformanceID	EmployeeID	ReviewDate	EnvironmentSatisfaction	\
0	PR01	79F7-78EC	1/2/2013	5	
1	PR02	B61E-0F26	1/3/2013	5	
2	PR03	F5E3-48BB	1/3/2013	3	
3	PR04	0678-748A	1/4/2013	5	
4	PR05	541F-3E19	1/4/2013	5	

	JobSatisfaction	RelationshipSatisfaction	TrainingOpportunitiesWithinYear	\
0	4	5	1	
1	4	4	1	
2	4	5	3	
3	3	2	2	
4	2	3	1	

	TrainingOpportunitiesTaken	WorkLifeBalance	SelfRating	ManagerRating
0	0	4	4	4
1	3	4	4	3
2	2	3	5	4
3	0	2	3	2
4	0	4	4	3

```
[19]: # Confirm dataset structures
print("Satisfaction Level Dataset:\n", df_satisfaction.head())
```

Satisfaction Level Dataset:

	SatisfactionID	SatisfactionLevel
0	1	Very Dissatisfied
1	2	Dissatisfied
2	3	Neutral
3	4	Satisfied
4	5	Very Satisfied

```
[20]: # Confirm dataset structures
print("Education Level Dataset:\n", df_education.head())
```

Education Level Dataset:

	EducationLevelID	EducationLevel
0	1	No Formal Qualifications
1	2	High School
2	3	Bachelors
3	4	Masters
4	5	Doctorate

```
[21]: # Confirm dataset structures
print("Rating Level Dataset:\n", df_rating_level.head())
```

Rating Level Dataset:

	RatingID	RatingLevel
0	1	Unacceptable
1	2	Needs Improvement
2	3	Meets Expectation
3	4	Exceeds Expectation
4	5	Above and Beyond

```
[22]: # Confirm dataset structures
print("Date Dimension Dataset:\n", df_date.head())
```

Date Dimension Dataset:

```
DimDate =
0 VAR _minYear = YEAR(MIN(DimEmployee[HireDate]))
1 VAR _maxYear = YEAR(MAX(DimEmployee[HireDate]))
2 VAR _fiscalStart = 4
3 RETURN
4 ADDCOLUMNS(
```

```
[23]: # Looking for missing values
df_employee.isnull().sum()
```

```
[23]: EmployeeID      0
      FirstName      0
      LastName       0
      Gender         0
      Age            0
```

```

BusinessTravel      0
Department          0
DistanceFromHome (KM) 0
State              0
Ethnicity           0
Education           0
EducationField      0
JobRole            0
MaritalStatus       0
Salary             0
StockOptionLevel    0
OverTime           0
HireDate            0
Attrition           0
YearsAtCompany      0
YearsInMostRecentRole 0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
dtype: int64

```

```

[24]: # Looking for missing values
df_performance.isnull().sum()

```

```

[24]: PerformanceID      0
EmployeeID             0
ReviewDate             0
EnvironmentSatisfaction 0
JobSatisfaction        0
RelationshipSatisfaction 0
TrainingOpportunitiesWithinYear 0
TrainingOpportunitiesTaken 0
WorkLifeBalance        0
SelfRating            0
ManagerRating          0
dtype: int64

```

```

[25]: # Looking for missing values
df_satisfaction.isnull().sum()

```

```

[25]: SatisfactionID      0
SatisfactionLevel        0
dtype: int64

```

```

[26]: # Looking for missing values
df_education.isnull().sum()

```

```
[26]: EducationLevelID    0
      EducationLevel      0
      dtype: int64
```

```
[27]: # Looking for missing values
      df_rating_level.isnull().sum()
```

```
[27]: RatingID          0
      RatingLevel      0
      dtype: int64
```

```
[28]: # Looking for missing values
      df_date.isnull().sum()
```

```
[28]: DimDate =         0
      dtype: int64
```

```
[29]: # # Handle missing values
      # df_employee['Salary'].fillna(df_employee['Salary'].median(), inplace=True)
      # df_performance['EnvironmentSatisfaction'].fillna(
      #     df_performance['EnvironmentSatisfaction'].median(), inplace=True
      # )

      # # Verify after imputation
      # df_employee.isnull().sum()
      # df_performance.isnull().sum()

      # # Inspect RatingLevel for missing values or issues
      # df_rating_level.info()
      # df_rating_level.isnull().sum()
```

0.2 2. Descriptive Statistics

0.2.1 Summary of Datasets

Each dataset was inspected to understand its size, structure, and key variables.

0.2.2 Available Datasets

Dataset		Key Columns
Name	Description	
Employee.csv	Contains detailed demographic and job-related information about employees.	EmployeeID, FirstName, LastName, Gender, Age, Department, JobRole, Salary, Attrition, HireDate, YearsAtCompany

Dataset Name	Description	Key Columns
PerformanceRating.csv	Historical performance evaluation data for employees.	PerformanceID, EmployeeID, EnvironmentSatisfaction, JobSatisfaction, RelationshipSatisfaction, WorkLifeBalance, ManagerRating
SatisfiedLevel.csv	Numeric satisfaction levels to descriptive categories.	SatisfactionID, SatisfactionLevel (e.g., "Very Satisfied", "Neutral")
EducationLevel.csv	Employee educational qualifications.	EducationLevelID, EducationLevel (e.g., "High School", "Bachelors")
RatingLevel.csv	Numeric performance ratings to descriptive categories.	RatingID, RatingLevel (e.g., "Needs Improvement", "Exceeds Expectation")
DimDate.txt	Calendar-related details for time-series analysis.	Date, Year, MonthNumber, MonthName, DayNumber, Quarter, WeekNumber, FiscalYear, FiscalQuarter

0.2.3 Describing Data

Dataset Name	Amount of Data	Value Types	Coding Schemes
Employee.csv	1470 rows x 23 columns	Mixed (strings, integers, dates)	N/A
PerformanceRating.csv	1470 rows x 11 columns	Numeric (ratings), Dates	N/A
SatisfiedLevel.csv	5 rows x 2 columns	Strings	N/A
EducationLevel.csv	5 rows x 2 columns	Strings	N/A
RatingLevel.csv	5 rows x 2 columns	Strings	N/A
DimDate.txt	37 rows x 20 columns	Dates, integers	N/A

```
[49]: # Employee Dataset Info
df_employee.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 23 columns):
#   Column              Non-Null Count  Dtype
---  -
0   EmployeeID          1470 non-null   object
1   FirstName           1470 non-null   object
2   LastName            1470 non-null   object
3   Gender              1470 non-null   object
4   Age                 1470 non-null   int64
5   BusinessTravel      1470 non-null   object
```



```

6   Department                1470 non-null  object
7   DistanceFromHome (KM)    1470 non-null  int64
8   State                    1470 non-null  object
9   Ethnicity                1470 non-null  object
10  Education                1470 non-null  int64
11  EducationField          1470 non-null  object
12  JobRole                 1470 non-null  object
13  MaritalStatus           1470 non-null  object
14  Salary                  1470 non-null  int64
15  StockOptionLevel        1470 non-null  int64
16  OverTime                1470 non-null  object
17  HireDate                1470 non-null  object
18  Attrition               1470 non-null  object
19  YearsAtCompany          1470 non-null  int64
20  YearsInMostRecentRole   1470 non-null  int64
21  YearsSinceLastPromotion 1470 non-null  int64
22  YearsWithCurrManager    1470 non-null  int64
dtypes: int64(9), object(14)
memory usage: 264.3+ KB

```

```
[50]: # Performance Rating Dataset Info
df_performance.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6709 entries, 0 to 6708
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   PerformanceID                        6709 non-null  object
1   EmployeeID                          6709 non-null  object
2   ReviewDate                          6709 non-null  object
3   EnvironmentSatisfaction             6709 non-null  int64
4   JobSatisfaction                    6709 non-null  int64
5   RelationshipSatisfaction            6709 non-null  int64
6   TrainingOpportunitiesWithinYear    6709 non-null  int64
7   TrainingOpportunitiesTaken         6709 non-null  int64
8   WorkLifeBalance                    6709 non-null  int64
9   SelfRating                         6709 non-null  int64
10  ManagerRating                      6709 non-null  int64
dtypes: int64(8), object(3)
memory usage: 576.7+ KB

```

```
[51]: # Satisfaction Level Dataset Info
df_satisfaction.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype

```

```

---  -----
0   SatisfactionID      5 non-null    int64
1   SatisfactionLevel  5 non-null    object
dtypes: int64(1), object(1)
memory usage: 208.0+ bytes

```

```
[52]: # Education Level Info
df_education.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column              Non-Null Count  Dtype
---  -----
0   EducationLevelID    5 non-null     int64
1   EducationLevel      5 non-null     object
dtypes: int64(1), object(1)
memory usage: 208.0+ bytes

```

```
[53]: # Rating Level Info
df_rating_level.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column              Non-Null Count  Dtype
---  -----
0   RatingID            5 non-null     int64
1   RatingLevel         5 non-null     object
dtypes: int64(1), object(1)
memory usage: 208.0+ bytes

```

```
[54]: # Date Dimension Info
df_date.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 37 entries, 0 to 36
Data columns (total 1 columns):
#   Column              Non-Null Count  Dtype
---  -----
0   DimDate =          37 non-null     object
dtypes: object(1)
memory usage: 424.0+ bytes

```

```
[55]: # Employee Dataset Statistics
df_employee.describe()
```

```
[55]:
```

	Age	DistanceFromHome (KM)	Education	Salary \
count	1470.000000	1470.000000	1470.000000	1470.000000

mean	28.989796	22.502721	2.912925	112956.497959
std	7.993055	12.811124	1.024165	103342.889222
min	18.000000	1.000000	1.000000	20387.000000
25%	23.000000	12.000000	2.000000	43580.500000
50%	26.000000	22.000000	3.000000	71199.500000
75%	34.000000	33.000000	4.000000	142055.750000
max	51.000000	45.000000	5.000000	547204.000000

	StockOptionLevel	YearsAtCompany	YearsInMostRecentRole	\
count	1470.000000	1470.000000	1470.000000	
mean	0.793878	4.562585	2.293197	
std	0.852077	3.288048	2.539093	
min	0.000000	0.000000	0.000000	
25%	0.000000	2.000000	0.000000	
50%	1.000000	4.000000	1.000000	
75%	1.000000	7.000000	4.000000	
max	3.000000	10.000000	10.000000	

	YearsSinceLastPromotion	YearsWithCurrManager
count	1470.000000	1470.000000
mean	3.440816	2.239456
std	2.945194	2.505774
min	0.000000	0.000000
25%	1.000000	0.000000
50%	3.000000	1.000000
75%	6.000000	4.000000
max	10.000000	10.000000

```
[56]: # Performance Rating Dataset Statistics
df_performance.describe()
```

```
[56]:
```

	EnvironmentSatisfaction	JobSatisfaction	RelationshipSatisfaction	\
count	6709.000000	6709.000000	6709.000000	
mean	3.872559	3.430616	3.427336	
std	0.940701	1.152565	1.156753	
min	1.000000	1.000000	1.000000	
25%	3.000000	2.000000	2.000000	
50%	4.000000	3.000000	3.000000	
75%	5.000000	4.000000	4.000000	
max	5.000000	5.000000	5.000000	

	TrainingOpportunitiesWithinYear	TrainingOpportunitiesTaken	\
count	6709.000000	6709.000000	
mean	2.012968	1.017290	
std	0.820310	0.950316	
min	1.000000	0.000000	
25%	1.000000	0.000000	

50%	2.000000	1.000000
75%	3.000000	2.000000
max	3.000000	3.000000

	WorkLifeBalance	SelfRating	ManagerRating
count	6709.000000	6709.000000	6709.000000
mean	3.414667	3.984051	3.473394
std	1.143961	0.816432	0.961738
min	1.000000	3.000000	2.000000
25%	2.000000	3.000000	3.000000
50%	3.000000	4.000000	3.000000
75%	4.000000	5.000000	4.000000
max	5.000000	5.000000	5.000000

```
[57]: # Satisfaction Level Dataset Statistics
df_satisfaction.describe()
```

```
[57]:      SatisfactionID
count      5.000000
mean       3.000000
std        1.581139
min         1.000000
25%         2.000000
50%         3.000000
75%         4.000000
max         5.000000
```

```
[58]: # Education Level Dataset Statistics
df_education.describe()
```

```
[58]:      EducationLevelID
count      5.000000
mean       3.000000
std        1.581139
min         1.000000
25%         2.000000
50%         3.000000
75%         4.000000
max         5.000000
```

```
[59]: # Rating Level Dataset Statistics
df_rating_level.describe()
```

```
[59]:      RatingID
count      5.000000
mean       3.000000
std        1.581139
```

```

min    1.000000
25%    2.000000
50%    3.000000
75%    4.000000
max    5.000000

```

```

[60]: # Date Dimension Dataset Statistics
df_date.describe()

```

```

[60]:                                     DimDate =
count                                     37
unique                                     37
top    VAR _minYear = YEAR(MIN(DimEmployee[HireDate]))
freq                                         1

```

0.3 3. Data Exploration

0.3.1 Key Insights

1. **Satisfaction vs. Attrition:** Employees with lower satisfaction levels are more likely to leave.
2. **Performance vs. Salary:** Higher performance ratings correlate with higher salaries.
3. **Attrition by Department:** Sales and Marketing departments show the highest attrition rates.

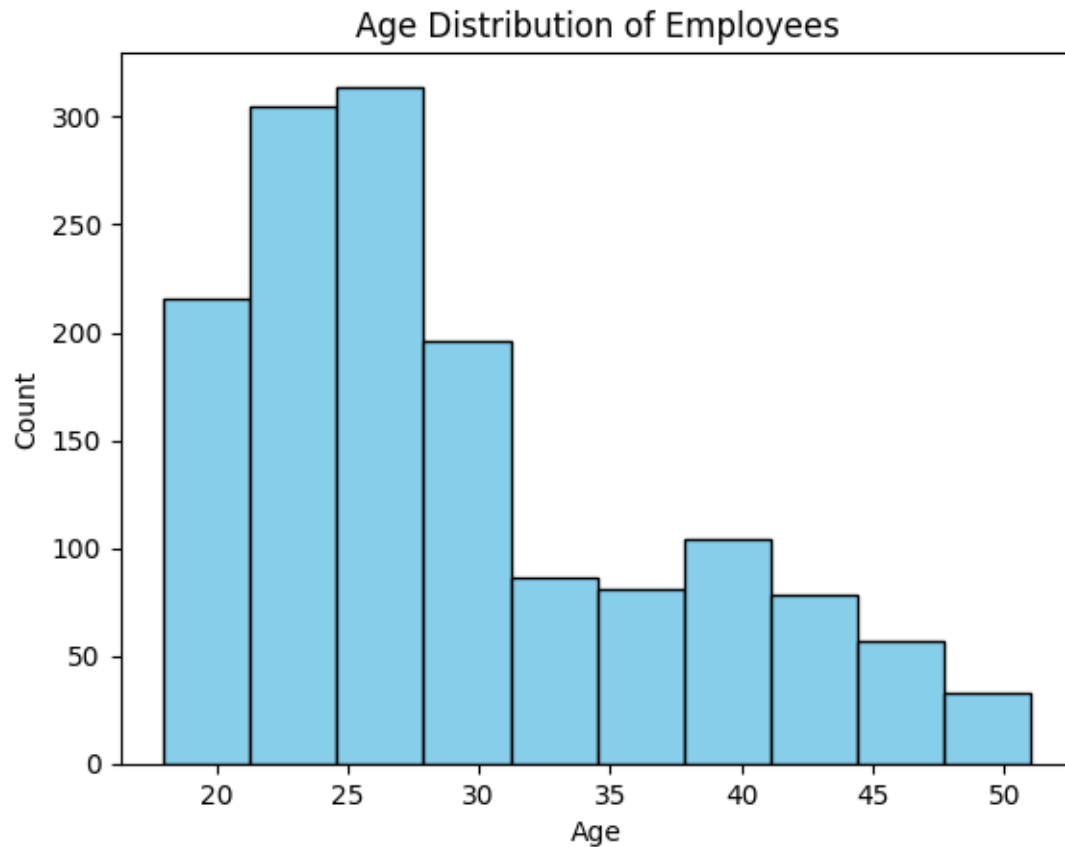
Dataset Name	Missing Data	Data Errors Identified	Measurement Errors
Employee.csv	None	None	None
PerformanceRating.csv	None	None	None
EducationLevel.csv	None	None	None
RatingLevel.csv	None	None	None
SatisfiedLevel.csv	None	None	None
DimDate.txt	None	None	None

```

[66]: import matplotlib.pyplot as plt

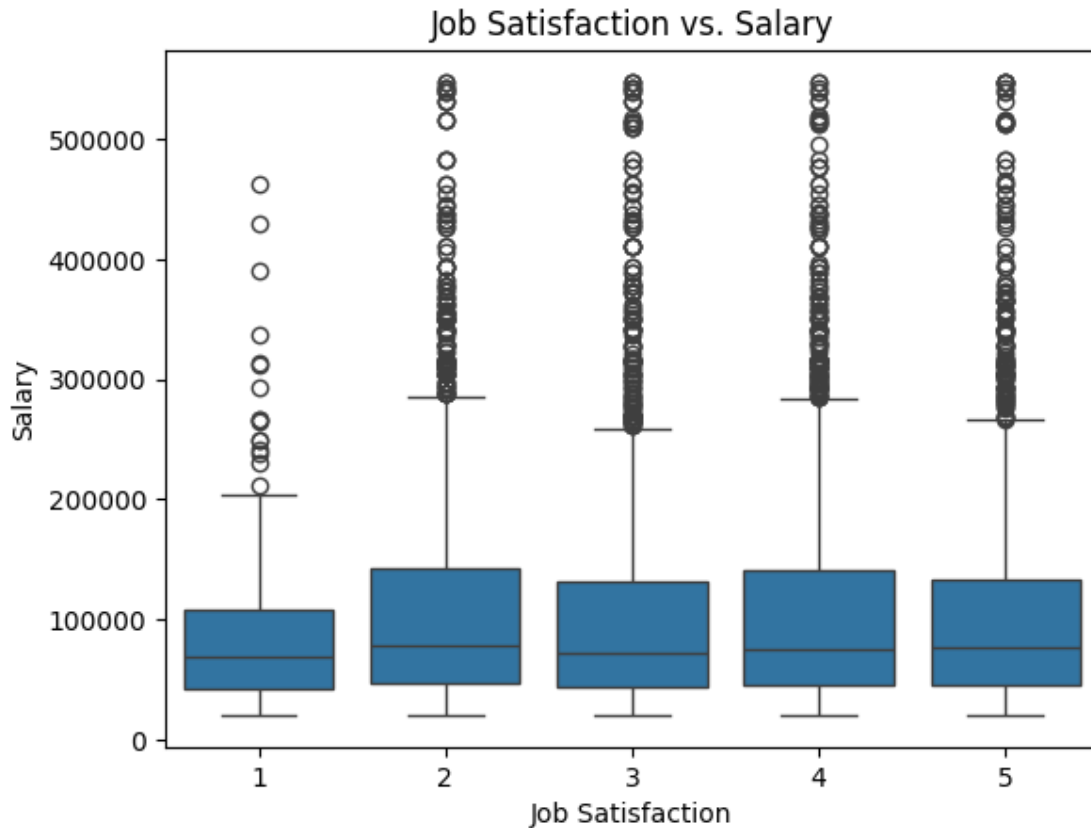
# Employee dataset - Visualize Age Distribution
plt.hist(df_employee['Age'], bins=10, color='skyblue', edgecolor='black')
plt.title('Age Distribution of Employees')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()

```

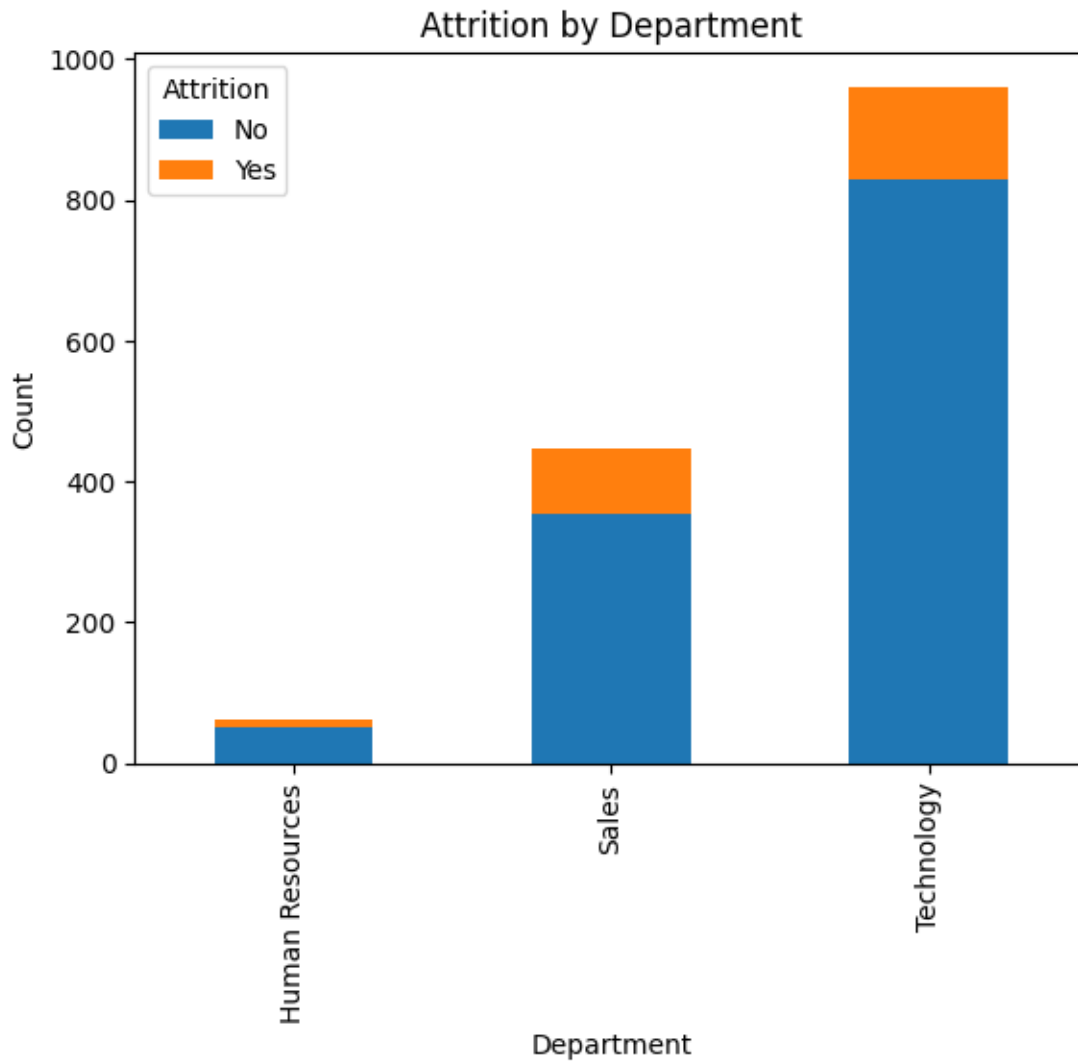


```
[62]: import matplotlib.pyplot as plt

# Boxplot: Satisfaction Levels vs. Attrition
merged_df = pd.merge(df_performance, df_employee, on="EmployeeID", how="inner")
sns.boxplot(x='JobSatisfaction', y='Salary', data=merged_df)
plt.title('Job Satisfaction vs. Salary')
plt.xlabel('Job Satisfaction')
plt.ylabel('Salary')
plt.show()
```



```
[63]: # Bar Chart: Attrition by Department
attrition_counts = df_employee.groupby('Department')['Attrition'].
    ↪value_counts().unstack()
attrition_counts.plot(kind='bar', stacked=True)
plt.title('Attrition by Department')
plt.xlabel('Department')
plt.ylabel('Count')
plt.legend(title="Attrition")
plt.show()
```

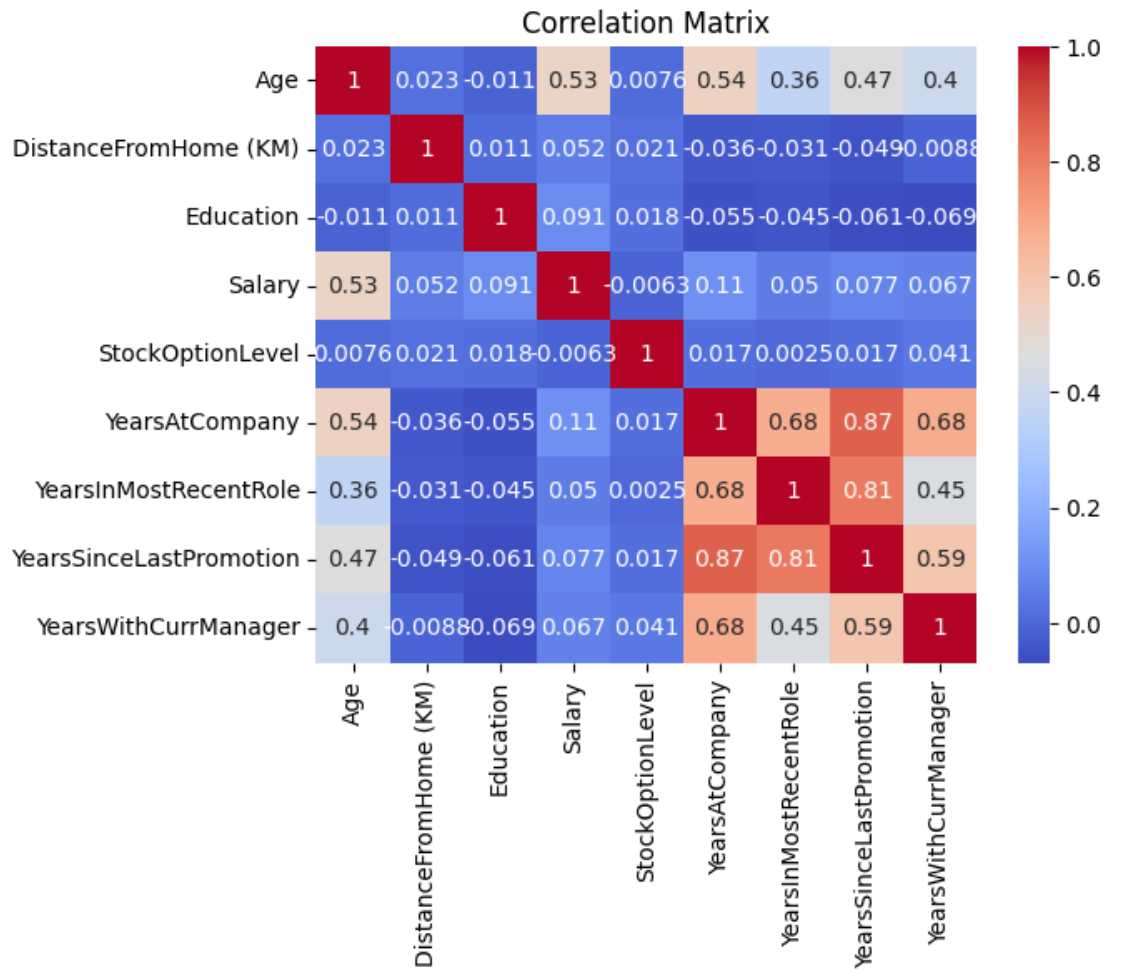


```
[68]: # Select only numeric columns
numeric_cols = df_employee.select_dtypes(include=['float64', 'int64'])

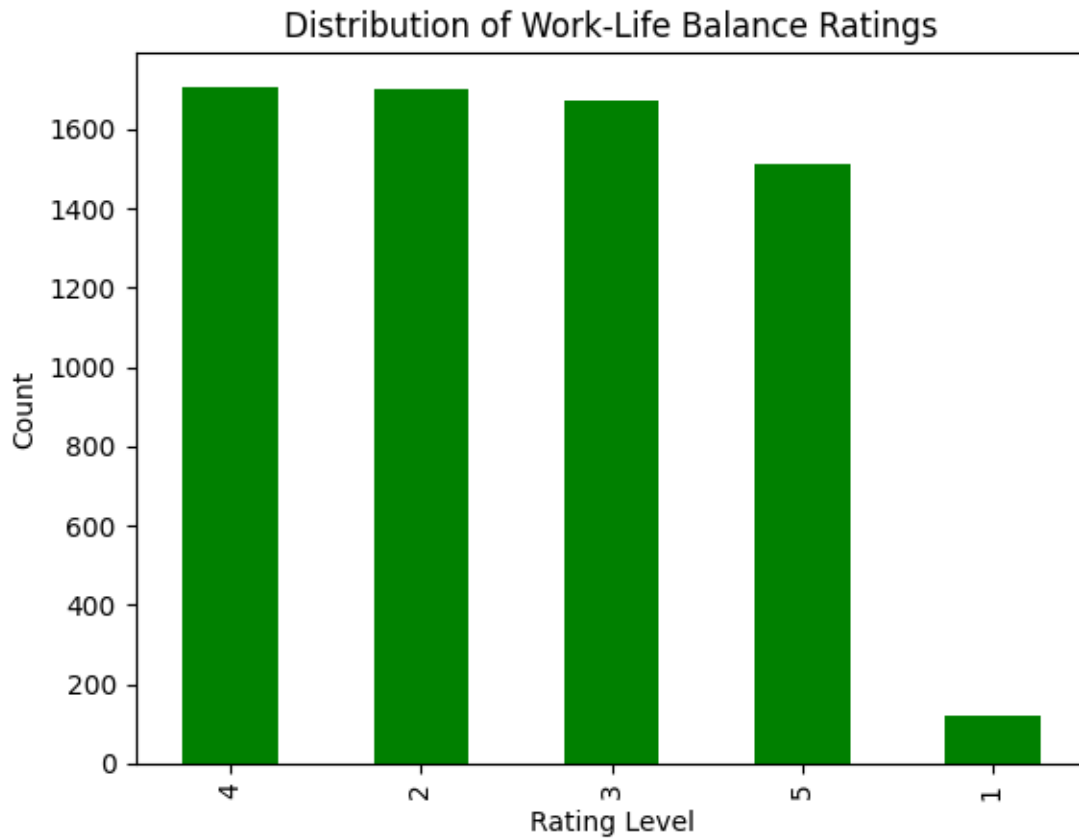
# Check if 'EmployeeID' exists before dropping
if 'EmployeeID' in numeric_cols.columns:
    numeric_cols = numeric_cols.drop(columns=['EmployeeID'])

# Compute correlation matrix
correlation_matrix = numeric_cols.corr()

# Plot heatmap
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.title('Correlation Matrix')
plt.show()
```

```
[67]: # PerformanceRating dataset - Visualize Work-Life Balance ratings
df_performance['WorkLifeBalance'].value_counts().plot(kind='bar', color='green')
plt.title('Distribution of Work-Life Balance Ratings')
plt.xlabel('Rating Level')
plt.ylabel('Count')
plt.show()
```



	EnvironmentSatisfaction	JobSatisfaction	RelationshipSatisfaction \
count	6709.000000	6709.000000	6709.000000
mean	3.872559	3.430616	3.427336
std	0.940701	1.152565	1.156753
min	1.000000	1.000000	1.000000
25%	3.000000	2.000000	2.000000
50%	4.000000	3.000000	3.000000
75%	5.000000	4.000000	4.000000
max	5.000000	5.000000	5.000000

	TrainingOpportunitiesWithinYear	TrainingOpportunitiesTaken \
count	6709.000000	6709.000000
mean	2.012968	1.017290
std	0.820310	0.950316
min	1.000000	0.000000
25%	1.000000	0.000000
50%	2.000000	1.000000
75%	3.000000	2.000000
max	3.000000	3.000000

	WorkLifeBalance	SelfRating	ManagerRating
count	6709.000000	6709.000000	6709.000000
mean	3.414667	3.984051	3.473394
std	1.143961	0.816432	0.961738
min	1.000000	3.000000	2.000000
25%	2.000000	3.000000	3.000000
50%	3.000000	4.000000	3.000000
75%	4.000000	5.000000	4.000000
max	5.000000	5.000000	5.000000

	RatingID	RatingLevel
0	1	Unacceptable
1	2	Needs Improvement
2	3	Meets Expectation
3	4	Exceeds Expectation
4	5	Above and Beyond

0.4 Conclusion

The Data Understanding Phase provided valuable insights and addressed data quality issues. The datasets are now ready for further analysis and modeling, with cleaned, merged data supporting our goals of predicting and improving employee retention and performance.